Advanced Machine Learning, ETH Zurich

Sampling with Langevin dynamics

From random steps to annealed Langevin sampling

Carlos Cotrini

November 23 and 26, 2026

Entry 1

Setup

We want

$$x_1, x_2, \ldots, x_n \sim p^\star$$



x_1



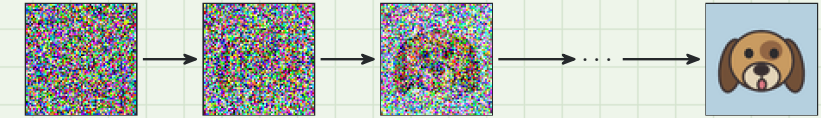
x_2

$\ldots$

but $p^\star$ is unknown.

Idea: Yield $X_T \sim p^\star$ by sampling $X_0 \sim \mathcal{N}(0, I)$ and then repeatedly applying

$$X_{t+1} = X_t + f(X_t, t) + \varepsilon \quad \text{with } \varepsilon \sim \mathcal{N}(0, I).$$



Goal: Find f satisfying this.

Entry 2

Itô calculus

Generalize $X_0, X_1, \ldots, X_T$ to a function X mapping $t \in [0, T]$ to some random variable $X(t)$. We must have

$$X(t + 1) = X(t) + f(X(t), t) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$



We generalize to any $\Delta \in (0, 1]$.

$$X(t+\Delta) = X(t) + f(X(t), t)\,\Delta + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \Delta\, I)$$

We then let $\Delta \to 0$ to obtain the desired function X.

$$X(t+\Delta) - X(t) = f\big(X(t), t\big)\,\Delta + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \Delta I)$$

$\Delta \to 0$

$$dX = f(X, t)\,dt + dW$$

We characterize the resulting function with this SDE, and say that

$$dW \sim \mathcal{N}(0, dt\, I)\,.$$

Entry 3

The Fokker-Planck equation

Let $q(x,t)$ be the pdf of $X(t)$, for $x \in \mathbb{R}^d$.

$$\frac{\partial q}{\partial t} = -\nabla \cdot J = -\sum_{i=1}^{d} \frac{\partial J_i}{\partial x_i}$$

with

$$J_i(x,t) = \underbrace{f_i(x,t)\, q(x,t)}_{\substack{\text{Amount of mass moved} \\ \text{by } f \text{ along } x_i}} \underbrace{-\frac{1}{2}\frac{\partial q(x,t)}{\partial x_i}}_{\substack{\text{Amount of mass moved} \\ \text{by } W \text{ along } x_i}}$$

Remember, we want to find f so that at the end of $[0, T]$, $X(T) \sim p^\star$.

$$\text{So, } X(t) \text{ stabilizes at } T \quad \Rightarrow \quad \frac{\partial q(x,t)}{\partial t} = 0.$$

So, to ensure $X(T) \sim p^\star$, we need

$$\frac{\partial q(x,t)}{\partial t} = 0 \quad \text{for } q = p^\star.$$



Zhang Lu, *Laozi Riding an Ox*,
Ming dynasty

$$J = 0 \quad \nabla \cdot J = 0$$

孰能濁以靜之徐清？
孰能安以久動之徐生？

Who can (make) the muddy water (clear)?
Let it be still, and it will gradually become clear.
Who can secure the condition of rest?
Let movement go on, and the condition of rest will gradually arise.

Laozi, *Tao Te Ching*, chapter 15. Translated by James Legge (1891).

If $J = 0$ then $\dfrac{\partial q}{\partial t} = -\nabla \cdot J = 0$ for $q = p^\star$.

$$\Rightarrow \quad J = f(x,t)\, q(x,t) - \frac{1}{2}\nabla_x q(x,t) = 0$$

$$\Rightarrow \quad f(x,t) = \frac{1}{2}\frac{\nabla_x q(x,t)}{q(x,t)} = \frac{1}{2}\nabla_x \log q(x,t)$$

$$\Rightarrow \quad \boxed{f(x,t) = \frac{1}{2}\nabla_x \log p^\star(x)} \quad !!$$

Entry 4

Back to the generalization

$$X(t+\Delta) = X(t) + f\big(X(t),t\big)\,\Delta + \sqrt{\Delta}\,\varepsilon_t \quad \text{with } \varepsilon_t \sim \mathcal{N}(0, I)$$

If $f(x,t) = \frac{1}{2}\,\nabla_x \log p^\star(x)$, then

$$\text{as } \Delta \to 0, \quad X(T) \sim p^\star \text{ for a large } T.$$

New problem: We need

$$\frac{1}{2} \nabla_x \log p^\star(x).$$

But $p^\star$ is unknown: we only have samples $x_1, x_2, \ldots, x_n \sim p^\star$.

Entry 5

Score matching

We learn it with a NN s_θ, so that $s_\theta(x) \approx \nabla_x \log p^\star(x)$, the score of $p^\star$, and take $f = \frac{1}{2}\, s_\theta$.

Collect $x_1, x_2, \ldots, x_n \sim p^\star$ and then train θ to minimize

$$J(\theta) = \frac{1}{2}\, \mathbb{E}_{x \sim p^\star}\left[\left\| s_\theta(x) - \nabla_x \log p^\star(x) \right\|^2 \right].$$

Hyvärinen's trick:

$$J(\theta) = \mathbb{E}_{x\sim p^\star}\Big[\frac{1}{2}\left\|s_\theta(x)\right\|^2 + \nabla\cdot s_\theta(x)\Big] + C$$

where C does not depend on θ.

NCSM, noise conditional score matching. Let $\tilde{X} = X + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, and $p_{\tilde{X}}$ the pdf of $\tilde{X}$:

$$\tilde{X} \to X \quad \text{as } \sigma \to 0.$$

$$\begin{aligned} J(\theta) &= \frac{1}{2}\,\mathbb{E}_{\tilde{X}\sim p_{\tilde{X}}}\left[\left\| s_\theta(\tilde{X}, \sigma) - \nabla \log p_{\tilde{X}}(\tilde{X})\right\|^2\right] \\ &= \frac{1}{2}\,\mathbb{E}_{X,\varepsilon}\left[\left\| s_\theta(X + \varepsilon, \sigma) + \frac{\varepsilon}{\sigma^2}\right\|^2\right] + C \text{ !!} \end{aligned}$$

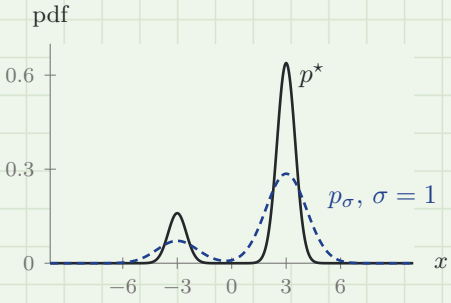
Algorithm 2: Training s_θ via NCSM

Input: data $x_1, \dots, x_n \sim p^\star$, a noise level $\sigma > 0$, a learning rate $\eta > 0$

1: **while** not converged **do**
2: Sample a minibatch $x^{(1)}, \dots, x^{(B)}$ of the data
3: Draw $\varepsilon^{(b)} \sim \mathcal{N}(0, \sigma^2 I)$ and set $\tilde{x}^{(b)} = x^{(b)} + \varepsilon^{(b)}$, for $b = 1, \dots, B$
4: $\hat{J}(\theta) = \dfrac{1}{2B} \displaystyle\sum_{b=1}^{B} \left\| s_\theta(\tilde{x}^{(b)}, \sigma) + \frac{\varepsilon^{(b)}}{\sigma^2} \right\|^2$
5: $\theta \leftarrow \theta - \eta \, \nabla_\theta \hat{J}(\theta)$
6: **end while**
7: **return** θ

Entry 6

Annealed Langevin sampling



an example on the line: $p^\star$ has two bumps

Remember: Algorithm 2 trains

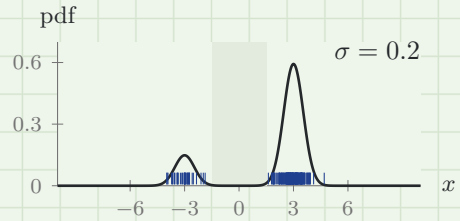
$$s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x),$$

where p_σ is the pdf of $\tilde{X} = X + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$.

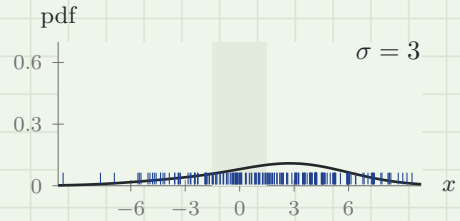
So the steps of page 10 with $f = \frac{1}{2}\, s_\theta(\cdot, \sigma)$, the Langevin steps, give $X(T) \sim p_\sigma$, a blur of $p^\star$.

$p_\sigma \to p^\star$ as $\sigma \to 0$.

New problem: which σ?

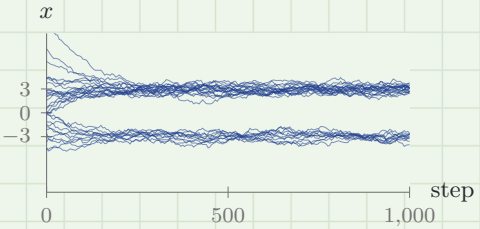


$p_\sigma \approx p^\star$. But 0 of 200 noisy points fall between the bumps: there s_θ is poor, and the steps rarely cross.

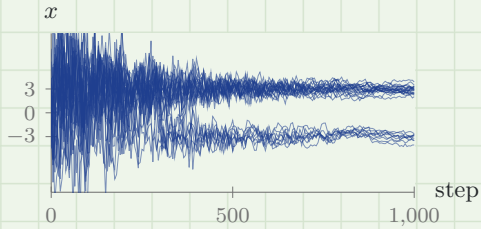


49 of 200 fall between the bumps: s_θ is good everywhere, and the steps move freely. But p_σ is a blur of $p^\star$.

Idea: Start at a large σ_1 and lower it step by step, $\sigma_1 > \sigma_2 > \cdots > \sigma_K > 0$. At each level σ_k, run the steps with $f = \frac{1}{2}\, s_\theta(\cdot, \sigma_k)$ from where the last level ended.



Only σ_K: 50 % end in the left bump.



Annealed: 21 %, and $p^\star$ puts 20 % there **!!**

The example of page 16, with the exact score of p_σ in place of s_θ: 2000 runs from $\mathcal{N}(0, \sigma_1^2)$, 30 of them drawn; σ from 4 down to 0.1 in 10 levels of 100 steps.

We train one network $s_\theta(x, \sigma)$ for all the levels, with $\varepsilon \sim \mathcal{N}(0, \sigma_k^2 I)$:

$$J(\theta) = \sum_{k=1}^{K} \lambda(\sigma_k) \underbrace{\frac{1}{2}\,\mathbb{E}_{X,\varepsilon}\Big[\Big\| s_\theta(X+\varepsilon, \sigma_k) + \frac{\varepsilon}{\sigma_k^2}\Big\|^2\Big]}_{\text{NCSM at } \sigma_k,\ \text{page } 14}$$

with $\lambda(\sigma) = \sigma^2$. Since

$$\sigma^2 \Big\| s_\theta + \frac{\varepsilon}{\sigma^2}\Big\|^2 = \Big\| \sigma\, s_\theta + \frac{\varepsilon}{\sigma}\Big\|^2 \quad \text{and} \quad \frac{\varepsilon}{\sigma} \sim \mathcal{N}(0, I),$$

every level weighs about the same.

Input: data $x_1, \dots, x_n \sim p^\star$, noise levels $\sigma_1 > \cdots > \sigma_K > 0$, a learning rate $\eta > 0$

1: **while** not converged **do**
2: Sample a minibatch $x^{(1)}, \dots, x^{(B)}$ of the data
3: Sample a level σ uniformly from $\{\sigma_1, \dots, \sigma_K\}$
4: Draw $\varepsilon^{(b)} \sim \mathcal{N}(0, \sigma^2 I)$ and set $\tilde{x}^{(b)} = x^{(b)} + \varepsilon^{(b)}$, for $b = 1, \dots, B$
5: $$\hat{J}(\theta) = \frac{\sigma^2}{2B} \sum_{b=1}^{B} \left\| s_\theta(\tilde{x}^{(b)}, \sigma) + \frac{\varepsilon^{(b)}}{\sigma^2} \right\|^2$$
6: $\theta \leftarrow \theta - \eta \nabla_\theta \hat{J}(\theta)$
7: **end while**
8: **return** θ

At level σ_k, we run the steps of page 10 with $f = \frac{1}{2}\, s_\theta(\cdot, \sigma_k)$:

$$X(t + \Delta_k) = X(t) + \frac{1}{2}\, s_\theta(X(t), \sigma_k)\, \Delta_k + \sqrt{\Delta_k}\, \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, I)$$

with the step length

$$\Delta_k = \Delta\, \frac{\sigma_k^2}{\sigma_K^2},$$

long at high noise, and $\Delta_K = \Delta$ at the last level.

The kicks $\sqrt{\Delta_k}\, \varepsilon_t$ are in proportion to σ_k, the least width of the bumps of p_{σ_k}.

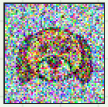
Input: a network $s_\theta(x, \sigma)$ trained by Algorithm 3, noise levels $\sigma_1 > \cdots > \sigma_K > 0$, a step length $\Delta > 0$, a number of steps T_k per level

Sample $X \sim \mathcal{N}(0, \sigma_1^2 I)$
for $k = 1, \ldots, K$ **do**
    $\Delta_k \leftarrow \Delta\, \sigma_k^2 / \sigma_K^2$
    for $t = 1, \ldots, T_k$ **do**
        Sample $\varepsilon \sim \mathcal{N}(0, I)$
        $X \leftarrow X + \frac{1}{2}\, s_\theta(X, \sigma_k)\, \Delta_k + \sqrt{\Delta_k}\, \varepsilon$
    end for
end for
return X ▷ approximately a sample of $p^\star$

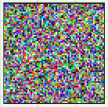
Entry 7

Back to diffusion models

Remember: to train a diffusion model, take an image, add noise for a random number of steps, and predict the noise that the noisy image holds.



noisy image $\tilde{x} = x + \varepsilon$



its noise ε/σ

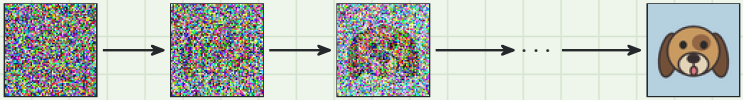
Algorithm 3 does the same, with a level σ at random. Its loss at σ weighs $\|\sigma\, s_\theta + \varepsilon/\sigma\|^2$ (page 19). Let $\varepsilon_\theta = -\sigma\, s_\theta$:

$$\left\|\sigma\, s_\theta(\tilde{x}, \sigma) + \frac{\varepsilon}{\sigma}\right\|^2 = \Big\| \underbrace{\frac{\varepsilon}{\sigma}}_{\text{the noise}} - \underbrace{\varepsilon_\theta(\tilde{x}, \sigma)}_{\text{its prediction}} \Big\|^2$$

Training a diffusion model is Algorithm 3, with $s_\theta = -\varepsilon_\theta/\sigma$. **!!**

A diffusion model also fades the image before it adds the noise; with the faded images in place of x, the same holds.

Remember: a diffusion model generates backward, from pure noise to an image, removing the predicted noise step by step: the idea of page 2.



Algorithm 4 does the same, from σ_1 down to σ_K. With $s_\theta = -\varepsilon_\theta/\sigma_k$, its step is

$$X \leftarrow X \underbrace{- \frac{\Delta_k}{2\sigma_k}\,\varepsilon_\theta(X, \sigma_k)}_{\text{remove part of the predicted noise}} \underbrace{+ \sqrt{\Delta_k}\,\varepsilon}_{\text{add fresh noise}}$$

The backward step of a diffusion model, DDPM, has the same two parts, and also undoes the fading.